

Experiment 'p_14_4_merge_sort_lexicographic_seq' Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 2

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_14_4_merge_sort_lexicographic_seq

Natural language statement: null

Method signature: p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
- multiset(inputs) == multiset(sorted)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)`

The method should respect the following contract:

`ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset
↪ (inputs) == multiset(sorted)`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPpadx3N7055LnejUh4czMKJfBWe

Model: gpt-5.5-2026-04-23

Created at: 1780669738

Finish reason: stop

Usage: 314 tokens in, and 2289 tokens out

```
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  {
    var x := inputs[n];
    var old := sorted;
    var k := 0;

    while k < |old| && old[k] <= x
      invariant 0 <= k <= |old|
      invariant old == sorted
      invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
      invariant forall i :: 0 <= i < k ==> old[i] <= x
    {
      k := k + 1;
    }
  }
}
```

```

if k < |old| {
  assert !(old[k] <= x);
  assert x <= old[k];
}

sorted := old[..k] + [x] + old[k..];

assert forall i, j | 0 <= i < j < |sorted| ensures sorted[i] <= sorted[j]
{
  if j < k {
    assert sorted[i] == old[i];
    assert sorted[j] == old[j];
  } else if j == k {
    assert i < k;
    assert sorted[i] == old[i];
    assert sorted[j] == x;
  } else if i < k {
    assert k < j;
    assert sorted[i] == old[i];
    assert sorted[j] == old[j - 1];
    assert 0 <= i < j - 1 < |old|;
  } else if i == k {
    assert sorted[i] == x;
    assert sorted[j] == old[j - 1];
    assert k <= j - 1 < |old|;
    if k < |old| {
      assert x <= old[k];
      if k < j - 1 {
        assert old[k] <= old[j - 1];
      }
    }
  } else {
    assert k < i;
    assert sorted[i] == old[i - 1];
    assert sorted[j] == old[j - 1];
    assert 0 <= i - 1 < j - 1 < |old|;
  }
}

assert old == old[..k] + old[k..];
assert sorted == old[..k] + [x] + old[k..];
assert multiset(sorted) == multiset(old[..k]) + multiset([x]) + multiset(old[k..]);
assert multiset(old) == multiset(old[..k]) + multiset(old[k..]);
assert multiset(sorted) == multiset(old) + multiset([x]);
assert inputs[..n + 1] == inputs[..n] + [inputs[n]];
assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([inputs[n]]);

n := n + 1;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(13,8): Error: invalid Ident
|
13 | var old := sorted;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(16,18): Error: openparen
  ↪ expected
|
16 | while k < |old| && old[k] <= x
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(16,26): Error: openparen
  ↪ expected
|
16 | while k < |old| && old[k] <= x
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(17,6): Error: closeparen
  ↪ expected
|
17 | invariant 0 <= k <= |old|
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(17,30): Error: openparen
  ↪ expected
|
17 | invariant 0 <= k <= |old|
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(18,20): Error: openparen
  ↪ expected
|
18 | invariant old == sorted
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,6): Error: closeparen
  ↪ expected
|
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,48): Error: openparen
  ↪ expected
|
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
| ^
```

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,57): Error: openparen
  ↪ expected
  |
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(19,67): Error: openparen
  ↪ expected
  |
19 | invariant forall i, j :: 0 <= i < j < |old| ==> old[i] <= old[j]
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(20,6): Error: closeparen
  ↪ expected
  |
20 | invariant forall i :: 0 <= i < k ==> old[i] <= x
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(20,46): Error: openparen
  ↪ expected
  |
20 | invariant forall i :: 0 <= i < k ==> old[i] <= x
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(21,4): Error: closeparen
  ↪ expected
  |
21 | {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(25,15): Error: openparen
  ↪ expected
  |
25 | if k < |old| {
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(26,18): Error: openparen
  ↪ expected
  |
26 | assert !(old[k] <= x);
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(26,27): Error: closeparen
  ↪ expected
  |
26 | assert !(old[k] <= x);
  | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(27,21): Error: openparen
  ↪ expected
  |

```

```

27 | assert x <= old[k];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(27,24): Error: closeparen
    ↪ expected
    |
27 | assert x <= old[k];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(30,17): Error: openparen
    ↪ expected
    |
30 | sorted := old[..k] + [x] + old[k..];
    | ^

19 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPqTDa8hMLXyDrqdSJj5tXzS9jnl

Model: gpt-5.5-2026-04-23

Created at: 1780669793

Finish reason: stop

Usage: 2985 tokens in, and 2289 tokens out

```

{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  {
    var x := inputs[n];
    var prev := sorted;
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|
      invariant prev == sorted
      invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
      invariant forall i :: 0 <= i < k ==> prev[i] <= x
    {
      k := k + 1;
    }
  }
}

```

```

if k < |prev| {
  assert !(prev[k] <= x);
  assert x <= prev[k];
}

sorted := prev[..k] + [x] + prev[k..];

assert forall i, j | 0 <= i < j < |sorted| ensures sorted[i] <= sorted[j]
{
  if j < k {
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j];
  } else if j == k {
    assert i < k;
    assert sorted[i] == prev[i];
    assert sorted[j] == x;
  } else if i < k {
    assert k < j;
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i < j - 1 < |prev|;
  } else if i == k {
    assert sorted[i] == x;
    assert sorted[j] == prev[j - 1];
    assert k <= j - 1 < |prev|;
    if k < |prev| {
      assert x <= prev[k];
      if k < j - 1 {
        assert prev[k] <= prev[j - 1];
      }
    }
  } else {
    assert k < i;
    assert sorted[i] == prev[i - 1];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i - 1 < j - 1 < |prev|;
  }
}

assert prev == prev[..k] + prev[k..];
assert sorted == prev[..k] + [x] + prev[k..];
assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
assert multiset(sorted) == multiset(prev) + multiset([x]);
assert inputs[..n + 1] == inputs[..n] + [inputs[n]];
assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([inputs[n]]);

n := n + 1;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(72,6): Error: invalid
```

```
  ↪ AssertStmt
```

```
  |  
72 | n := n + 1;  
  | ^
```

1 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

```
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
```

```
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPrO8YITgjCUWYUUqvSLigqjil2i

Model: gpt-5.5-2026-04-23

Created at: 1780669850

Finish reason: stop

Usage: 2939 tokens in, and 1505 tokens out

```
{  
  sorted := [];  
  var n := 0;  
  while n < |inputs|  
    invariant 0 <= n <= |inputs|  
    invariant multiset(sorted) == multiset(inputs[..n])  
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]  
  {  
    var x := inputs[n];  
    var prev := sorted;  
    var k := 0;  
  
    while k < |prev| && prev[k] <= x  
      invariant 0 <= k <= |prev|  
      invariant prev == sorted  
      invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]  
      invariant forall i :: 0 <= i < k ==> prev[i] <= x  
    {  
      k := k + 1;  
    }  
  
    if k < |prev| {  
      assert !(prev[k] <= x);  
      assert x <= prev[k];  
    }  
  
    sorted := prev[..k] + [x] + prev[k..];  
  }
```

```

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] <= sorted[j]
{
  if j < k {
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j];
  } else if j == k {
    assert i < k;
    assert sorted[i] == prev[i];
    assert sorted[j] == x;
  } else if i < k {
    assert k < j;
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i < j - 1 < |prev|;
  } else if i == k {
    assert sorted[i] == x;
    assert sorted[j] == prev[j - 1];
    assert k <= j - 1 < |prev|;
    if k < |prev| {
      assert x <= prev[k];
      if k < j - 1 {
        assert prev[k] <= prev[j - 1];
      }
    } else {
      assert false;
    }
  } else {
    assert k < i;
    assert sorted[i] == prev[i - 1];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i - 1 < j - 1 < |prev|;
  }
}

assert prev == prev[..k] + prev[k..];
assert sorted == prev[..k] + [x] + prev[k..];
assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
assert multiset(sorted) == multiset(prev) + multiset([x]);
assert inputs[..n + 1] == inputs[..n] + [inputs[n]];
assert multiset(inputs[..n + 1]) == multiset(inputs[..n]) + multiset([inputs[n]]);

n := n + 1;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(4,0): Error: a postcondition
  ↳ could not be proved on this return path
|
4 | {
|   ^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(3,26): Related location:
  ↳ this is the postcondition that could not be proved
|
3 |   ensures multiset(inputs) == multiset(sorted)
|   ^^

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(27,6): Error: assertion
  ↳ might not hold
|
27 | assert x <= prev[k];
|     ^^^^^^

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
  ↳ Dafny body, including the curly braces that surround it. Do not show the signature
  ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPsVpwTvMmEKdkFYp33JY8VWxDis

Model: gpt-5.5-2026-04-23

Created at: 1780669919

Finish reason: stop

Usage: 4035 tokens in, and 8296 tokens out

```

{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    decreases |inputs| - n
  {
    var x := inputs[n];
    var prev := sorted;
    assert multiset(prev) == multiset(inputs[..n]);
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|

```

```

invariant prev == sorted
invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
invariant forall i :: 0 <= i < k ==> prev[i] <= x
decreases |prev| - k
{
  k := k + 1;
}

if k < |prev| {
  assert !(prev[k] <= x);
  assert x <= prev[k] || prev[k] <= x;
  assert x <= prev[k];
}

sorted := prev[..k] + [x] + prev[k..];

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] <= sorted[j]
{
  if j < k {
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j];
  } else if j == k {
    assert i < k;
    assert sorted[i] == prev[i];
    assert sorted[j] == x;
  } else if i < k {
    assert k < j;
    assert sorted[i] == prev[i];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i < j - 1 < |prev|;
  } else if i == k {
    assert sorted[i] == x;
    assert sorted[j] == prev[j - 1];
    assert k <= j - 1 < |prev|;
    assert k < |prev|;
    assert x <= prev[k];
    if k < j - 1 {
      assert prev[k] <= prev[j - 1];
    }
  } else {
    assert k < i;
    assert sorted[i] == prev[i - 1];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i - 1 < j - 1 < |prev|;
  }
}

assert prev == prev[..k] + prev[k..];
assert sorted == prev[..k] + [x] + prev[k..];
assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);

```

```

    assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
    assert multiset(sorted) == multiset(prev) + multiset([x]);
    assert x == inputs[n];
    assert multiset(sorted) == multiset(inputs[..n]) + multiset([inputs[n]]);
    assert inputs[..(n + 1)] == inputs[..n] + [inputs[n]];
    assert multiset(inputs[..(n + 1)]) == multiset(inputs[..n]) + multiset([inputs[n]]);
    assert multiset(sorted) == multiset(inputs[..(n + 1)]);

    n := n + 1;
}

assert n == |inputs|;
assert inputs[..n] == inputs;
assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq/program.dfy(30,6): Error: assertion
  ↪ might not hold
|
30 | assert x <= prev[k] || prev[k] <= x;
| ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPwfzU4K8cYSrfS5IgWqlA4I6FJs

Model: gpt-5.5-2026-04-23

Created at: 1780670177

Finish reason: length

Usage: 4948 tokens in, and 5348 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```

method p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<
  ↪ string>)

```

```

    ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    ensures multiset(inputs) == multiset(sorted)
{
  sorted := [];
  var n := 0;
  while n < |inputs|
    invariant 0 <= n <= |inputs|
    invariant multiset(sorted) == multiset(inputs[..n])
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    decreases |inputs| - n
  {
    var x := inputs[n];
    var prev := sorted;
    assert multiset(prev) == multiset(inputs[..n]);
    var k := 0;

    while k < |prev| && prev[k] <= x
      invariant 0 <= k <= |prev|
      invariant prev == sorted
      invariant forall i, j :: 0 <= i < j < |prev| ==> prev[i] <= prev[j]
      invariant forall i :: 0 <= i < k ==> prev[i] <= x
      decreases |prev| - k
    {
      k := k + 1;
    }

    if k < |prev| {
      assert !(prev[k] <= x);
      assert x <= prev[k] || prev[k] <= x;
      assert x <= prev[k];
    }

    sorted := prev[..k] + [x] + prev[k..];

    forall i, j | 0 <= i < j < |sorted|
      ensures sorted[i] <= sorted[j]
    {
      if j < k {
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j];
      } else if j == k {
        assert i < k;
        assert sorted[i] == prev[i];
        assert sorted[j] == x;
      } else if i < k {
        assert k < j;
        assert sorted[i] == prev[i];
        assert sorted[j] == prev[j - 1];
        assert 0 <= i < j - 1 < |prev|;
      } else if i == k {
        assert sorted[i] == x;
      }
    }
  }
}

```

```

    assert sorted[j] == prev[j - 1];
    assert k <= j - 1 < |prev|;
    assert k < |prev|;
    assert x <= prev[k];
    if k < j - 1 {
        assert prev[k] <= prev[j - 1];
    }
} else {
    assert k < i;
    assert sorted[i] == prev[i - 1];
    assert sorted[j] == prev[j - 1];
    assert 0 <= i - 1 < j - 1 < |prev|;
}
}

assert prev == prev[..k] + prev[k..];
assert sorted == prev[..k] + [x] + prev[k..];
assert multiset(sorted) == multiset(prev[..k]) + multiset([x]) + multiset(prev[k..]);
assert multiset(prev) == multiset(prev[..k]) + multiset(prev[k..]);
assert multiset(sorted) == multiset(prev) + multiset([x]);
assert x == inputs[n];
assert multiset(sorted) == multiset(inputs[..n]) + multiset([inputs[n]]);
assert inputs[..(n + 1)] == inputs[..n] + [inputs[n]];
assert multiset(inputs[..(n + 1)]) == multiset(inputs[..n]) + multiset([inputs[n]]);
assert multiset(sorted) == multiset(inputs[..(n + 1)]);

n := n + 1;
}

assert n == |inputs|;
assert inputs[..n] == inputs;
assert multiset(inputs) == multiset(sorted);
}

```

Total Token Usage

Input tokens: 15221

Output tokens: 19727

Reasoning tokens: 16509

Sum of 'total tokens': 34948

Experiment Timings

Overall Experiment started at 1780669738465, ended at 1780670325100, lasting 586635ms (586.64 seconds)

Iteration #4 started at 1780669917767, ended at 1780670159571, lasting 241804ms (241.80 seconds)

Iteration #5 started at 1780670159572, ended at 1780670325099, lasting 165527ms (165.53 seconds)

Iteration #1 started at 1780669738466, ended at 1780669791997, lasting 53531ms (53.53 seconds)

Iteration #2 started at 1780669791998, ended at 1780669849799, lasting 57801ms (57.80 seconds)

Iteration #3 started at 1780669849799, ended at 1780669917766, lasting 67967ms (67.97 seconds)